

# The wall gradient is the warm start: why projected neural predictions slow a RANS solver down

Ali Jabbary\*

Department of Mechanical Engineering, Urmia University, Urmia, Iran

Kasra Ghanavati

School of Computing and Mathematical Sciences,  
University of Greenwich, London, UK

August 30, 2026

Draft — not submitted

**Status:** full draft, 2026-08-30. **All planned measurements are in** — the thirteen-case generality sweep and the exclusive wall-clock run both landed today. Every number here is traceable to a `results/*.json` file and a script. **No number in this file may be changed without changing PLANS.md too.**

**The framing, decided 2026-08-30.** This paper is *not* a speedup paper. The largest number in this literature is  $26.3\times$ ; ours is  $+18.4\%$ , and a paper that competes on that axis loses before it is read. What we have that nobody else has is a **measured, checkable reason** why a surrogate stored the way the entire neural-operator field stores its outputs is not merely a *worse* initial condition than a uniform freestream but a **catastrophic** one — and a single scalar, computable on your own mesh before you run anything, that says which it will be. The speedup is the confirmation that the criterion is the right one. Venue is left open; the draft is venue-neutral so that choice costs a formatting pass and nothing else.

## Abstract

Neural surrogates for external aerodynamics are usually evaluated as predictors. Used instead as *initial conditions* for a production RANS solver, they are routinely **worse than no initialisation at all** — and we show the reason is not accuracy but **representation**, that it is measurable in advance, and that fixing it turns a large loss into a reliable gain.

Take the **exact converged flow field**, store it the way neural-operator outputs are normally stored — a  $128^2$  Cartesian raster, or a wall-fitted  $256\times 64$  grid at equal budget — and hand it back to the solver as a starting point. Convergence of total drag is  **$-548\%$  and  $-173\%$**  against a uniform freestream. A *perfect* answer, resampled, costs the solve five times over. The cause is geometric: the first cell on a wall-resolved mesh is  $10^{-5}$  chords tall, no  $16,384$ -value grid has a station anywhere near it, and so the round trip does not degrade the near-wall state but **deletes** it — leaving  $\sim 1900\%$  error in the first-cell tangential velocity gradient, the quantity viscous drag integrates, against  $2851\%$  for a uniform freestream. Cartesian or body-fitted makes no difference; refining the raster does not help, because resolving one cell across the inner layer would need  $N \approx 11,800$ .

---

\*Corresponding author: `st_a.jabbary@urmia.ac.ir`. ORCID 0000-0003-0573-6909.

This gives a **criterion a practitioner can evaluate on their own mesh before running anything**: how much of the first-cell wall gradient does my representation retain? Ours, a Transolver-style point model queried directly at the solver’s cell centres with no resampling, retains 46% of it where a grid retains none. **The criterion is necessary and not sufficient, and we are precise about which**: every seed that loses the wall gradient costs the solve, without exception, so a practitioner can use it to *rule a representation out* before spending anything. Keeping the gradient does not by itself buy a saving — one arm in this study retains it perfectly and is still the worst of all — which is what the remaining two conditions are for.

Acting on the criterion yields a recipe with **three conditions, each isolated by a controlled arm that changes one variable on one prediction**, none sufficient alone: evaluate at the solver’s own **cell centres** (the same prediction resampled swings total drag by 93 points), hand over the **boundary layer only** (the whole field costs  $<-568\%$ ), and hand over **velocity and eddy viscosity together** (eddy viscosity alone is the study’s *best* arm on viscous drag and its *worst* on total drag, because Spalart–Allmaras production is strain-driven).

Applied across **thirteen cases** from  $2^\circ$  to  $12^\circ$ , attached flow into incipient separation, the recipe accelerates convergence of **viscous drag — 60–84% of the total drag here — by +18.4% [+12.4, +25.3], winning on 13 of 13 cases (exact sign test  $p = 0.0002$ )**, monotone across convergence bands, with a converged-field oracle control at +93.6% and a negative control statistically indistinguishable from no seed at all (+3.4%,  $p = 0.27$ ). Total drag is not readable over all thirteen — two cases converge to drag values 1% apart under different seeds — and we report that rather than excluding them. **The effect is smaller than we pre-registered (+30%) and more certain than we expected.**

Because a bad seed can be much worse than no seed, we add an acceptance test: run  $K = 25$  probe iterations, read one scalar from the residual history, and either continue or discard the seed and start cold. The rule never observes a cold run, its threshold is calibrated leave-one-case-out, and it bounds the worst case at  $(1 + K/N) \times \text{cold}$  by construction. Across 5 cases  $\times$  15 seeding strategies (70 seeds) it turns an ungated mean of -163.6% with a -1169.6% tail into **+1.5% mean, worst -5.8%, admitting none of the 46 harmful seeds**. The gate is insurance rather than a profit centre, and we report the one quantity it fails on (lift) rather than the four it handles.

We report three of our own falsified predictions, eight measurement rules — six of which changed a *sign* on our own data — one number we withdraw, and one pre-registered threshold we miss.

## 1. Introduction

A neural surrogate that predicts a flow field in 30 ms is not, by itself, a faster CFD pipeline. Engineering decisions are made on force coefficients with error bars, and a surrogate whose drag is 5% off does not replace a solve — it precedes one. The natural role for it is therefore as an **initial condition**: if the surrogate’s field is closer to the answer than a uniform freestream, the solver should need fewer iterations to get there, and the saving is free of any accuracy claim, because the converged answer is the solver’s, not the network’s.

That argument is clean and, as stated, wrong. This paper is about why, and about what has to be true instead.

Our starting configuration is deliberately unfavourable to shortcuts. The solver is unmodified OpenFOAM v2606 `simpleFoam` with the Spalart–Allmaras model. The mesh is a wall-resolved body-fitted C-grid, 31,700 cells, first cell centre  $5e-6$  chords off the wall, aspect ratios up to  $2e5$ . The Reynolds number is  $3e6$  — flight, not a demonstration. The surrogate is a Transolver-style point model trained on AirFRANS, evaluated on airfoils and angles it did not see. The metric is not a residual threshold: it is the number of iterations before a force coefficient enters and stays

inside a band around its converged value, because that is the quantity an engineer actually waits for.

Under those conditions the naive experiment fails, and it fails in a way that is easy to measure and easy to misattribute. Take one surrogate prediction, restricted to the boundary layer, and hand it to the solver twice: once evaluated at the solver’s own cell centres, once resampled through a wall-fitted 256x64 grid first. The first is **+33.9%** on total drag. The second is **-58.8%**. Same network, same weights, same case, same prediction — 93 percentage points, and the only difference is the representation it travelled through.

That is not an accuracy effect, and the cleanest way to see so is to remove accuracy from the experiment entirely. Resample the **exact converged field** and seed with that: -172.6% on a wall-fitted 256x64 grid, -548.4% on a Cartesian  $128^2$  grid. A perfect answer, stored the way neural-operator outputs are normally stored, is a *worse* initial condition than a uniform freestream.

The failure is specific and it has a location: the wall. Viscous drag is the integral of the tangential velocity gradient in the first cell off the surface, and it is 60–84% of the total drag in these cases. A 16,384-value grid, however it is arranged, has no sample point a few times  $1e-6$  chords from a wall — the finest wall-normal spacing 16,384 values can buy is orders of magnitude coarser. Projecting a field onto such a grid and back therefore does not *degrade* the near-wall state, it *deletes* it and replaces it with something near-freestream — overestimating wall shear by a factor of  $\sim 20$ . We show this is true even when the field being projected is the exact converged answer, which isolates the projection from any question about surrogate accuracy: the oracle’s own projection carries 1890% error in the wall gradient, and the trained network queried at the cell centres carries 54%.

## Contributions

Ordered by what we think survives, not by what is largest.

1. **A representational failure of the field’s standard output format, measured with the accuracy confound removed** (§6). The *exact converged answer*, stored as a  $128^2$  raster or a wall-fitted  $256 \times 64$  grid, is a catastrophically bad initial condition — -548% and -173% on drag convergence. This is not about any network’s accuracy, and it is not fixable by refining the raster (§7.1).
2. **A necessary criterion for it, computable before running a solve**: first-cell wall-gradient retention. Every seed that loses the gradient costs the solve, without exception — so it rules representations *out* for free. It does not rule them in: **nf\_mesh** retains the gradient perfectly and is the worst arm in the study, which is why conditions 2 and 3 exist. It is a property of the *representation*, not of the model (§6).
3. **A recipe that follows from the criterion, with three necessary conditions**, each isolated by a controlled arm changing exactly one variable on one prediction; none sufficient alone (§5.2–§5.4).
4. **Generality at  $n = 13$** , attached flow to incipient separation, pre-registered admission gate excluding nothing, +18.4% on viscous drag, 13/13,  $p = 0.0002$ , with a passing oracle control and a **null** negative control (§5.1).
5. **An acceptance certificate** that bounds the worst case at  $(1 + K/N) \times \text{cold}$  using a rule that never sees a cold run (§8).
6. **Three falsified predictions** a reader would otherwise make: seed the pressure (inert), seed what the cold solver is slow at (negative), seed the wake (worth +0.5%, against a  $26.3 \times$  claim elsewhere) (§7).
7. **A measurement protocol** of eight rules, six of which each changed a *sign* on our own data, under which we withdraw one of our own previously reported numbers and miss one of our own pre-registered thresholds (§4).

## Is this a fact about your network?

No, and that is the reason the oracle arms are in the study rather than in an appendix. Every statement about representation is reproduced with the **exact converged field** in place of the prediction: the projection of a perfect answer carries 1881% wall-gradient error and costs  $-206\%$  on drag. Nothing about that depends on which network produced the field, on how it was trained, or on how accurate it is. The surrogate supplies the *practical* version of the result; the oracle supplies the version that cannot be explained away by our model being bad.

## Is the cold baseline a straw man?

The industrial alternative to a uniform freestream is a potential-flow initialisation, and OpenFOAM ships one. We ran it as an arm: `potentialFoam` alone is worth  $+0.6\%$  on  $C_d@1\%$  and  $+3.3\%$  on lift — inert. A uniform freestream is therefore not a weak baseline being beaten by a strong method; it is what the strong classical alternative is also worth on this configuration.

## What this paper does not claim

**It does not claim a speed record, and it is not competing for one.** The largest number in this literature is  $26.3\times$ ; ours is  $+18.4\%$ . §7.3 explains with a measurement rather than a rebuttal why that number and ours are about different regimes — an oracle seed of the entire wake buys  $+0.5\%$  here. A reader who wants the fastest warm start should read that paper; a reader who wants to know whether *their* surrogate can warm-start *their* solver, and to find out before paying for it, should read this one.

It does not claim the surrogate is accurate — accuracy is a separate question and irrelevant here, since the solver converges to its own answer regardless. And it does not claim generality of the *demonstration* beyond 2-D incompressible steady RANS with one turbulence model on NACA 4-digit sections at one Reynolds number. The **mechanism** is more general than that by construction — no grid of 16,384 values has a station  $5\cdot 10^{-6}$  chords off a wall, whatever solver or turbulence model reads it — but we measured it in one place and §10 says so plainly.

## 2. Related work

**Neural surrogates for external aerodynamics.** AirfRANS established the reference dataset and the point-cloud evaluation protocol. Transolver and its successors, and point-cloud neural operators such as PCNO (arXiv:2501.14475), predict fields on the native mesh points rather than on a raster. That capability is usually presented as an accuracy or memory convenience. **We show it is the difference between a warm start that works and one that is worse than nothing** — a surrogate stored as a  $128^2$  image cannot be used this way at all, and the loss is not recoverable by refining the raster (§7.1).

**Warm-starting linear and nonlinear solvers with learning.** Learned initial guesses and preconditioners are an established line: Nows (arXiv:2511.02481) learns operator-aware warm starts; Spectrally Safe (arXiv:2606.21828) constrains learned corrections to preserve convergence; PCGBandit (arXiv:2509.08765) selects solver configurations online. NVIDIA’s hybrid initialisation (arXiv:2503.15766) is the closest in spirit — a network’s prediction as a CFD initial field — and reports gains on configurations where the near-wall state is not the bottleneck. Our contribution relative to this line is not the idea of seeding; it is (a) the demonstration that *how the prediction is represented* dominates *how accurate it is*, and (b) a per-case acceptance test.

**Wake initialisation.** The largest reported acceleration in this literature —  $26.3\times$  iterations,  $16.4\times$  wall-clock (arXiv:2501.14699) — comes from initialising the far wake. We deliberately seed

the opposite region. §7.3 reports an oracle experiment bounding what *any* wake model could buy in our configuration: **+0.5% on viscous drag**. That is a statement about our regime, not a criticism of theirs, and it is the reason our restriction to the boundary layer is a finding rather than a compromise.

**Fallbacks and guarantees.** Preserving worst-case behaviour by falling back to the classical method when a learned component is untrustworthy is not new; learning-augmented algorithms with dual warm starts (arXiv:2605.09382) apply exactly this pattern to linear assignment. **We cite it as prior art for the pattern.** What is ours is the instantiation for a PDE solver’s initial field: a decision rule that reads only a short probe of the solve it is about to commit to, a leave-one-case-out calibration, and a measured capture-versus-cost curve showing that longer probes are monotonically worse.

### 3. Setup

**Solver.** OpenFOAM v2606 (ESI), `simpleFoam`, steady incompressible SIMPLEC (`consistent yes`), Spalart–Allmaras. `nNonOrthogonalCorrectors 2`. Under-relaxation `U 0.7`, `nuTilda 0.4` — chosen, not defaulted; see §4 rule 2. Budget 6000 iterations. Every case is instrumented with both the `forceCoeffs` and `forces` function objects, so total, pressure and viscous drag are available separately at every iteration.

**Mesh.** Body-fitted C-grid generated by `blockMesh` from a specification we control (`solver/cgrid.py`): 31,700 cells, 20-chord far field, stitch-free wake cut via shared vertex ids, first cell  $1e-5$  chords tall (centre 5e-6), wall-normal grading to  $y^+ < 1$ . The same mesh is used for every arm of every experiment, so mesh quality is never a differential effect.

**Cases.** NACA 4-digit sections at  $Re = 3e6$ ,  $u_{inf} = 1$ , kinematic pressure. Two sets. The **mechanism study** is 5 cases (0012@4°, 2412@2°, 0015@6°, 0012@0°, 2415@5°) carrying fifteen seeding arms, which is where every controlled contrast and the acceptance gate are measured. The **generality corpus** is 13 cases (0012 at 8/10/12°, 2412 at 8/10°, 0018 at 4/8°, 4415 at 2/4°, 2415 at 2/8°, 0015 at 2/4°) carrying four arms, which is where the headline is measured. The two sets are disjoint, and every number states which it came from.

**Surrogate.** A Transolver-style point model,  $(B,N,7) \rightarrow (B,N,4)$ , trained on AirFRANS under the conventions of the companion paper (dimensional fields,  $nu = 1.56e-5$ ,  $Re$  from  $|u_{in}|$ ). It is queried directly at the C-grid cell centres — `solver/surrogate_seed.py` — with no intermediate representation of any kind. Its training `sdf` distribution is centred on 0.23 chords, which is why seeds are cut off at 3.5 chords and why the outer field is left cold.

**Arms.** Every experiment carries a **converged-field oracle** as a control. The oracle is the cold run’s own converged solution, re-injected as an initial condition, and it must post a large positive saving before any other arm in that experiment is read. Across twelve experiments the control has read +68% to +99.9%. An experiment whose control fails is a broken measurement, and we say so rather than reporting its other arms.

### 4. How to measure a warm start without fooling yourself

This section is a contribution, not preamble. Each of the first six rules below changed a **sign** on this project’s own data; rules 7 and 8 were added by the thirteen-case sweep and each cost us a result. All are implemented in `solver/scoring.py` or `scripts/reanalyse_depth.py`, each with a test that names the mistake it prevents.

1. **A residual threshold measures a convergence rate only while the residual is still falling.** Report every threshold as a multiple of the run’s residual floor and refuse to read

anything below  $\sim 5x$ . The same arm read +15%, +31% and +13% at 1.9x, 1.3x and 0.9x the floor — noise presented as a trend.

2. **The floor itself is usually an artifact, so find out which one.** Ours was under-relaxation at 0.9 with SIMPLEC, not the  $2e5$ -aspect-ratio cells:  $U\ 0.7 / \nu_{Tilda}\ 0.4$  moved the floor 30x, from  $1.1e-5$  to  $3.5e-7$ . Ten other variants — longer budgets, tighter inner tolerances, a better wake mesh — bought nothing.
3. **An arm that never reaches the target must be bounded at its full budget, not dropped.** Dropping non-finishers turned a true -199.4% into -31.2%: the failing arm was rewarded for failing.
4. **Score every arm against one external reference**, never against its own final value. Grading the oracle against itself turned a +73.5% control into +1.0%.
5. **Only arms whose coefficient has stopped moving may define that reference.** Peak-to-peak over the last tenth of the run, against a quarter of the band. A single diverged arm previously set the reference and condemned an entire table; restricting the reference to settled arms moved the spread from 3.104% to 0.334%.
6. **nNonOrthogonalCorrectors multiplies the pressure history.** Parse per Time block; zipping fields by index across a log made pressure lag velocity 3:1 and moved every shallow number.

One consequence of rule 5 is worth stating even though it is small here. Because the reference is a median over the arms that have settled, *adding* an arm to a tree re-scores every arm already in it. **A number must therefore name the arm set it was computed over**, and ours does (`-drop-arm` declares it). We measured the sensitivity rather than assuming it away: removing our most recently added arm moves every headline number by at most 0.5 percentage points.

**The primary metric.** `iterations_to_force_band`: the first iteration after which a coefficient stays within  $\pm b$  of the reference for the rest of the run. Saving is 1 - warm/cold, bounded at the budget. We report  $b = 1\%, 0.5\%, 0.2\%$  and state which bands are readable.

**Readability.** A band is readable only if the settled arms agree about the converged value to well inside it (rule 5) — concretely, to within half the band. On the core study the largest disagreement is 0.232% of Cd, so  $b = 1\%$  and  $b = 0.5\%$  are readable (limits 0.5% and 0.25%) and  $b = 0.2\%$  is not (limit 0.1%). **The 0.5% verdict is thin:** 0.232% against a 0.25% limit. We report the margin rather than only the verdict, because one additional case can flip it.

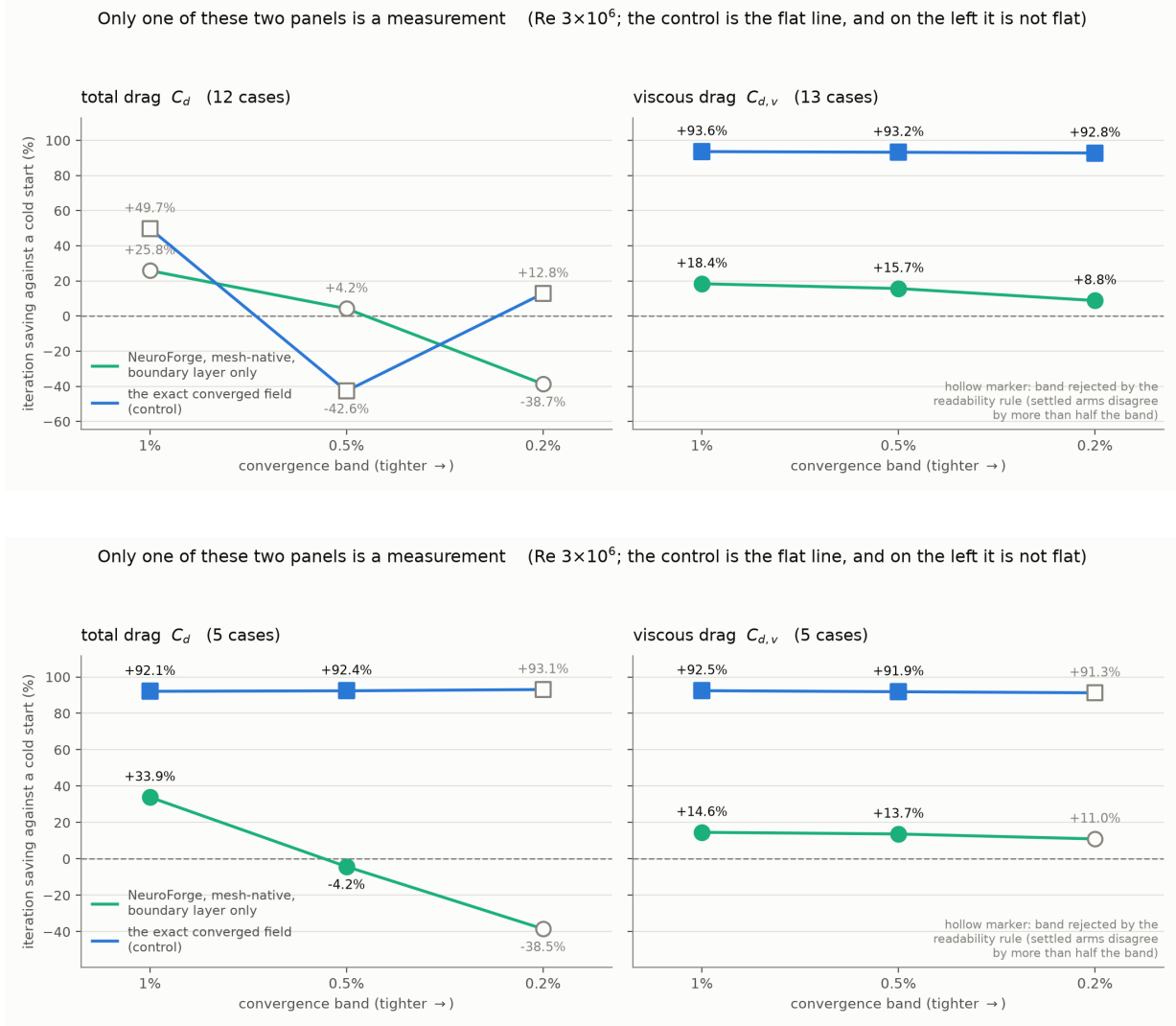
**Provisional.** Every readability verdict in this paper is a property of the case set, since the reference is a median over cases and arms. The 13-case sweep recomputes all of them, and they are re-checked rather than inherited.

**The saving depends on the band, and the two drag components differ.** On the five-case mechanism study:

| arm         | Cd@1%  | Cd@0.5% | Cd@0.2%                      | Cd_v@1% | Cd_v@0.5% | Cd_v@0.2%                    |
|-------------|--------|---------|------------------------------|---------|-----------|------------------------------|
| nf_bl       | +33.9% | -4.2%   | -38.5% ( <i>unreadable</i> ) | +14.6%  | +13.7%    | +11.0% ( <i>unreadable</i> ) |
| oracle_mesh | +92.1% | +92.4%  | +93.1%                       | +92.5%  | +91.9%    | +91.3%                       |

**Viscous drag is monotone across bands and total drag is not.** Monotone stability *is* the evidence that a number is a convergence-rate measurement rather than an artifact of where a wandering curve happens to cross a line — and Cd\_v is 60–84% of the drag here.

**Figure 2** (results/bands.png, from scripts/plot\_bands.py) draws the same two panels on the **thirteen-case corpus**, where the point is sharper still: every total-drag band is rejected by the readability rule *and the oracle control itself swings*  $+49.7\% \rightarrow -42.6\% \rightarrow +12.8\%$  across them, while on viscous drag the control is flat at  $+93\%$  and the trained arm is monotone. A control that is not flat is a measurement that is not a measurement. Rejected bands are plotted hollow rather than deleted, because a curve with its last point removed would look steadier than the measurement is. (results/bands\_n5.png is the same figure on the five-case study.)



**A withdrawal.** A previously recorded  $+41.8\%$  at  $C_d@0.5\%$  was read against a reference that a diverged arm had moved. On the settled reference over the declared arm set that row is  $-4.2\%$  — readable, and negative. The  $+33.9\%$  at  $C_d@1\%$  is unaffected on the five-case study, and §5.1 reports what happens to it at thirteen. We report this because a protocol that only ever deleted inconvenient numbers would not be a protocol.

**Rule 7, learned from the corpus: an admission threshold and a scoring threshold must be derived from the same quantity.** We pre-registered a gate that admits a case if its arms agree on final drag to within 2%, and a scoring rule that can read a 1% band only if they agree to within 0.5%. Both were declared in advance, and they disagree — so the sweep admitted

two cases it could not then read, and they cost us every total-drag row (§5.1). The gate should have been derived from the readability limit. We did not change it after the fact; we report the inconsistency, because a study can admit cases it cannot score and the failure is invisible until it happens.

**Rule 8, from the same sweep: a converged run is finished, not truncated.** OpenFOAM exits early when `residualControl` is satisfied. A scorer that judges completeness by run length alone calls that a truncation and discards it — penalising precisely the arms that converge *fastest*. Ours did, and it silently dropped the oracle control on one case, turning a +93% control into a +49.7% bound.

**Statistics.** Savings 1 - warm/cold are left-skewed, so we report percentile bootstrap 95% CIs (10,000 resamples) rather than t-intervals, plus an exact two-sided sign test. At  $n = 5$  the smallest attainable  $p$  is 0.0625; we state this wherever  $n < 6$  rather than reporting a non-significant  $p$  as if the test could have succeeded. The thirteen-case corpus of §5.1 is where the sign test can bite: 13/13 gives  $p = 0.0002$ .

## 5. The result, and the three conditions behind it

Two studies, disjoint (§3). The **corpus** — thirteen cases, four arms — carries the headline and answers *does this generalise*. The **mechanism study** — five cases, fifteen arms — carries the controlled contrasts and answers *why, and under what conditions*. §5.1 is the first; §5.2–§5.4 are the second.

### 5.1 The headline, at thirteen cases

`scripts/corpus_probe.py`. Thirteen cases — five NACA sections,  $2^\circ$  to  $12^\circ$ , from attached flow into incipient separation — each admitted under a gate declared before the sweep ran (residual floor  $\leq 1e-5$ , arms agreeing on final  $C_d$  to within 2%). **All thirteen were admitted**, so nothing here rests on a discretionary exclusion.

| row            | nf_bl         | 95% CI         | wins         | sign test         | oracle control | Cartesian control     |
|----------------|---------------|----------------|--------------|-------------------|----------------|-----------------------|
| <b>Cd_v@1%</b> | <b>+18.4%</b> | [+12.4, +25.3] | <b>13/13</b> | <b>p = 0.0002</b> | +93.6% (13/13) | +3.4% ( $p = 0.27$ )  |
| Cd_v@0.5%      | +15.7%        | [+10.6, +21.8] | 13/13        | $p = 0.0002$      | +93.2%         | +4.9% ( $p = 0.27$ )  |
| Cd_v@0.2%      | +8.8%         | [+4.3, +13.5]  | 11/13        | $p = 0.022$       | +92.8%         | -43.6% ( $p = 0.58$ ) |

Per case at the 1% band: **+3, +9, +9, +10, +11, +11, +16, +17, +19, +25, +28, +35, +47**. There is no losing case and no case carrying the mean.

Three properties make this a rate measurement rather than a crossing artifact: it is **monotone** across every band, the **oracle control** is flat at +93% and wins 13/13, and the **negative control** — the exact converged field resampled onto a  $128^2$  Cartesian grid — is statistically indistinguishable from no seed at all (+3.4%,  $p = 0.27$ ). A pipeline that manufactured savings would not produce a null there.

**Total drag, lift and pressure drag are unreadable over the thirteen**, and two cases carry all of it. `naca4415` at  $2^\circ$  and  $4^\circ$  have arms that settle on drag values **1.04% and 1.27% apart**, against  $\leq 0.113\%$  for every other case in the corpus. No arm is *unsettled* on those two — they settle in different places, which is the signature of a case without a unique steady fixed point, and the same signature `naca4412@3` showed. All three are thick cambered sections at low incidence, and all three have the corpus’s worst residual floors ( $8.75e-6$  and  $1.22e-6$ , against  $\sim 1e-7$  elsewhere).

**Sensitivity analysis, and it is post hoc.** Dropping those two cases leaves eleven, on which Cd@1% becomes readable and reads **+30.0%** [**+5.3, +47.0**], **10/11 wins, p = 0.012**, with the oracle control at +95.4% (11/11) and the Cartesian control at -262.5%. We report it because withholding it would be hiding a result, and we label it because the exclusion was chosen after seeing the data. It is not the headline. We do **not** carry the Cd@0.5% row from that subset: the oracle control there reads -13.9% with one case at -1112%, so by our own rule the row is unreadable regardless of what `nf_bl` does.

**Against our own pre-registered bar.** We committed in advance to  $\geq +30\%$  on a force-convergence metric, at flight Reynolds, on a wall-resolved mesh, from a trained model, with a passing oracle control, at  $n \geq 12$ . The primary analysis meets every clause except the first: **+18.4%, not +30%**. We are not restating the threshold now that we can see the result — that is exactly the move §4 exists to prevent. The honest summary is that the effect is smaller than we hoped for and more certain than we expected: thirteen cases out of thirteen,  $p = 0.0002$ , on the quantity that is 60–84% of the drag.

### The three conditions, and the study that isolates them

Everything from here to §5.6 is the **mechanism study**: `scripts/mesh_native_probe.py`, five cases, **one prediction**, one variable changed per arm, all twenty solves complete. Scored over the thirteen-arm `repr3` set with `oracle_wake` dropped (§4); including it moves no entry below by more than 0.5 points.

| arm                     | what it hands over                 | residual 5e-6 | <b>Cd@1%</b>   | Cl@1%         | Cd_v@1%       |
|-------------------------|------------------------------------|---------------|----------------|---------------|---------------|
| <code>nf_bl</code>      | u, v, nut in the BL, mesh-native   | < -80.5%      | <b>+33.9%</b>  | +10.1%        | +14.6%        |
| <code>nf_bl_proj</code> | the same, resampled through 256x64 | +22.1%        | <b>-58.8%</b>  | +25.4%        | +7.7%         |
| <code>nf_bl_nut</code>  | eddy viscosity only                | +1.2%         | <b>-293.2%</b> | <b>+41.1%</b> | <b>+42.4%</b> |
| <code>nf_bl_vel</code>  | velocity only                      | -8.2%         | -40.3%         | -10.3%        | -4.9%         |

The recommended arm in this table, `nf_bl`, is the one measured at thirteen cases in §5.1. Its Cd@1% here is +33.9%; §5.1 is where that number meets a corpus.

### 5.2 Condition 1 — evaluate at the solver’s cell centres

`nf_bl` and `nf_bl_proj` differ in exactly one respect: whether the identical prediction was resampled through a wall-fitted 256x64 grid before being written. Total drag swings **93 percentage points**, and the seed’s first-cell wall-gradient error rises from 54% to 1583% — landing in the same place as the matched oracle arm, `fitted_bl`, which is the *exact converged field* through the same 256x64 round trip and the same boundary-layer mask (1881%). The round trip destroys the wall gradient regardless of the quality of the field that entered it: a network prediction and a perfect answer come out of it indistinguishable where it matters.

The signs that go the other way confirm the mechanism rather than complicating it. Resampling *helps* the residual (+22.1%) and *helps* lift (+25.4%), because a resampled field is smoother — it changes less per iteration — and lift is pressure-dominated. **A study that scored residuals alone would have concluded the projection was the better seed.**

### 5.3 Condition 2 — hand over the boundary layer only

The region axis is controlled the same way as the resampling axis: both arms are the same network, both mesh-native, differing only in whether the handover is masked to the boundary layer. Cd@1%, five cases, cold = 805 iterations, oracle control **+92.1%**:

| arm (network, mesh-native) | region handed over  | Cd@1%               |
|----------------------------|---------------------|---------------------|
| nf_bl                      | boundary layer only | <b>+33.9%</b>       |
| nf_mesh                    | the whole field     | <b>&lt; -568.3%</b> |

Handing over the outer field fails because the model is extrapolating there — its training **sdf** distribution is centred on 0.23 chords while the C-grid reaches 20 — and an extrapolated outer field is an inconsistent boundary condition for a boundary layer the solver is still computing.

**Together with §5.2 this gives two controlled contrasts sharing a common arm (nf\_bl)**, one per axis, each changing a single variable. Neither factor alone is sufficient: mesh-native evaluation of the whole field is the worst arm in the study, and boundary-layer restriction under resampling is still negative.

We deliberately do **not** present these as a 2×2. The fourth corner — a network prediction of the whole field, resampled — was never run, and the arms that would fill it come from a different population (oracle rather than network). A table whose cells mix provenance would read as a design when it is a gap.

**The oracle bound on the resampled row**, which removes the accuracy confound entirely, is reported separately and is the stronger statement:

| arm           | field                           | representation             | Cd@1%   |
|---------------|---------------------------------|----------------------------|---------|
| fitted_256x64 | exact converged answer          | wall-fitted 256x64         | -172.6% |
| fitted_bl     | exact converged answer, BL only | wall-fitted 256x64         | -206.1% |
| cartesian_128 | exact converged answer          | Cartesian 128 <sup>2</sup> | -548.4% |

Even a perfect field, resampled, costs the solver more than starting it cold.

### 5.4 Condition 3 — velocity and eddy viscosity together

This condition was not anticipated; it came out of splitting the channels.

Eddy viscosity alone is simultaneously the **best** arm in the study on viscous drag (+42.4%) and lift (+41.1%), matching the oracle projection, and the **worst** on total drag (-293.2%). Velocity alone is bad at everything. Only the pair is positive on total drag.

The reason is the structure of the Spalart–Allmaras production term, which is driven by the strain rate. An eddy viscosity handed over without the velocity field that generated it is inconsistent with the strain the solver computes from the field it actually has; the momentum sink is therefore wrong, the pressure field must reorganise, and Cd\_p — 16–40% of the drag here — is destroyed. The shear-driven quantities do not care, and speed up.

**Consistency between handed-over channels is not a detail of the recipe. It is most of it**, and §7.3 finds the same lesson again at a completely different length scale.

## 5.5 A common-mode limitation, checked rather than assumed

`write_case` floors `nuTilda` at its freestream value, which clips 37% of the boundary-layer cells. That sounds like it could produce these swings. It cannot: the clipped cells hold values  $\sim 88\times$  below the peak, so the floor removes only **2.1–2.3%** of the eddy-viscosity field’s energy, and it applies identically to every arm including the oracle. It is a common-mode limitation of the study, not a differential effect that could move an arm from +33.9% to -293.2%.

## 5.6 The residual objection

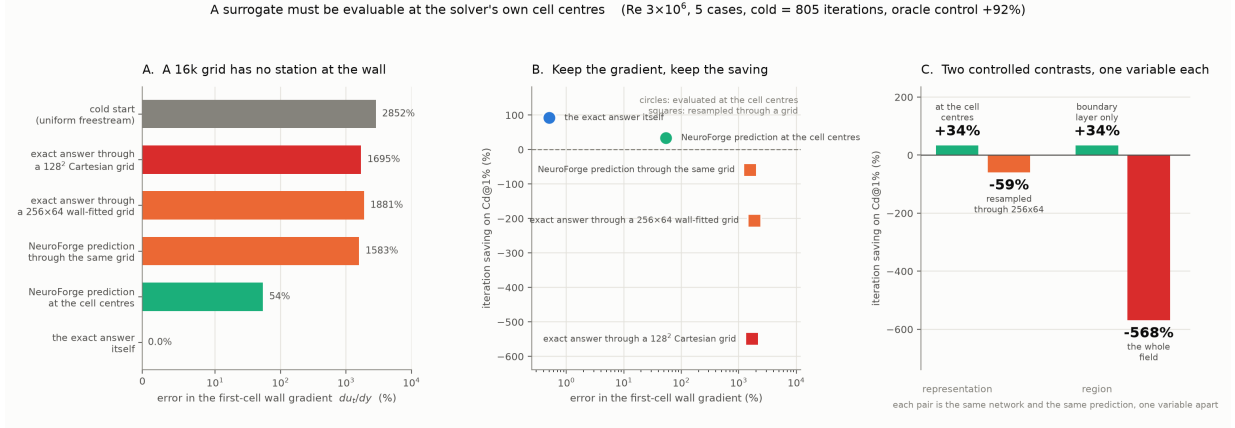
`nf_bl` is **negative on the residual at every depth** and positive on drag; `nf_bl_proj` is the exact inverse. A reviewer is right to look hard at that. Our answer has three parts.

1. **The residual is not the objective.** Nobody runs a RANS solve to obtain a small residual; they run it to obtain a force coefficient that has stopped moving. The residual is a proxy, and this study measures the proxy failing: one seed is +22.1% on the proxy and -58.8% on what the proxy stands for. We report both, always.
2. **We know what the residual is rewarding.** The `Ux` residual measures how much the field changes per iteration, so it rewards smoothness. A resampled field has had its near-wall structure interpolated away and therefore changes less, while carrying 1583% error in the wall gradient. The residual is not being fooled at random — it is faithfully measuring something that is not drag.
3. **The choice of metric was pre-committed and cuts against us.** The force metric replaced the residual metric for the reason in §4 rule 1, which predates this arm; and the readability rule then rejected rows we would rather have quoted, including the withdrawal above.

The honest residue stays in the paper: **on the residual, `nf_bl` is worse than a cold start.** §8 is what makes that survivable — the same 25-iteration gate that bounds drag bounds the residual metric’s worst case at -7.6%.

## 6. The mechanism

**Figure 1** (`results/mechanism.png`, from `scripts/plot_mechanism.py`) is the paper’s central figure and carries this section and §5 together. **A** — what each representation does to the first-cell wall gradient, including projections of the exact answer. **B** — that gradient error against what it bought on drag, one point per arm, circles evaluated at the cell centres and squares resampled; the two clusters separate cleanly. **C** — the two controlled contrasts of §5.2 and §5.3, each pair the same network and the same prediction, one variable apart.



`scripts/seed_gradient_diagnostic.py`, six cases. For each seed *as the solver received it*, the error in the first-cell tangential velocity gradient  $du_t/dy$  — the quantity viscous drag integrates — measured along the outward wall normal (not by nearest neighbour: near-wall cells here are  $\sim 2500\times$  wider than they are tall, so nearest-neighbour sampling would silently sample the wrong cell).

| seed                                                              | wall-gradient error | BL velocity error |
|-------------------------------------------------------------------|---------------------|-------------------|
| cold start (uniform freestream)                                   | 2851%               | 90.4%             |
| Cartesian $128^2$ projection of <b>the exact answer</b>           | 1695%               | 56.6%             |
| wall-fitted $256 \times 64$ projection of <b>the exact answer</b> | 1890%               | 51.0%             |
| <b>trained surrogate, queried at the cell centres</b>             | <b>54%</b>          | <b>15.2%</b>      |
| oracle (the exact answer itself)                                  | 0%                  | 0%                |

Read the middle two rows carefully: **the field being projected is the exact converged answer**. Any question of surrogate accuracy is removed. Both projections still overestimate the wall shear by a factor of  $\sim 20$ , because they place near-freestream velocity at a cell centre  $5e-6$  chords off the wall — no  $16,384$ -value grid has a station there. Against a cold start they remove between a third and a half of the error in the quantity that decides drag. The network, evaluated pointwise at the solver's own cell centres with no resampling at all, removes 98% of it.

This is not an accuracy story with a smoothness twist. An earlier version of this work hypothesised that projected seeds were *accurate but rough*, and that the solver preferred smooth. That hypothesis is dead: the projection is  $35\times$  **less** accurate on the wall gradient, not accurate-but-rough.

**The wall-fitted projection is the important control.** One might assume the problem is the Cartesian raster, and that a body-fitted  $256 \times 64$  output layout would fix it. It does not —  $1890\%$  versus  $1695\%$ , i.e. slightly worse — because at equal output budget ( $16,384$  values) a wall-fitted grid buys angular resolution, not wall-normal resolution, and wall-normal resolution is the entire question.

## 7. What does not work, and why that matters

Three predictions a careful reader would make, each falsified by measurement. We keep them because a recipe with three falsified neighbours is a recipe; one without them is an anecdote.

## 7.1 Refining the raster does not help

A uniform Cartesian seed fails at  $Re = 3e6$  at **any resolution we can reach**:  $128^2$  to  $421^2$  is flat. It is not a training problem — the arm under test is the exact converged answer. Resolving one cell across the inner layer would need  $N \sim 11,800$ , which is 28x beyond what AirfRANS itself stores.

We also tested the obvious dimensionless criterion.  $\delta/h$  (boundary-layer thickness over cell size) looks clean on a Reynolds sweep — a sign change at  $\delta/h = 2.0$  — and then fails on the grid axis. Across that sweep  $\delta/h$  moves 5x while the viscous ratio  $y(y+=30)/h$  collapses **1660x**. The near-wall viscous scale, not the boundary-layer thickness, is what decides whether a grid representation can carry a warm start.

Warm-starting at moderate Reynolds does work: +14.4% at  $Re = 1e4$ . A previously recorded  $Re = 1e3$  result mostly did not survive a parser fix (+58%  $\rightarrow$  +8.1%) and those runs sit 3–9x above their residual floor; they are excluded pending re-measurement on the relaxed settings [[B5]].

## 7.2 Seeding what the cold solver is slow at makes it slower

Iterations to settle within 1% of converged, cold versus seeded with the exact field:

| quantity             | cold  | oracle seed | share of Cd |
|----------------------|-------|-------------|-------------|
| viscous drag $Cd_v$  | ~700  | ~53         | 60–84%      |
| lift $C_l$           | ~950  | 1           | —           |
| pressure drag $Cd_p$ | ~1850 | 1–2         | 16–40%      |

A cold solver is slow at pressure and comparatively fast at the near-wall velocity gradient; a surrogate is the reverse. The obvious inference — hand over the pressure, keep the near-wall velocity — is what this project pursued for two working sessions, and it is **false**:

- **fitted\_p** (pressure only) is **inert**: +0.2% on drag and +0.1% elsewhere at every depth.
- **composite** (potential-flow pressure + surrogate boundary layer) is **negative**: -305.4%.
- **potential** (**potentialFoam** alone — the free industrial baseline) is inert on drag (+0.6%) and mildly positive on lift (+3.3%).

The reason is SIMPLE’s structure: pressure is *recomputed* from continuity given the velocity field, so a pressure seed inconsistent with  $U$  is overwritten within a few iterations. Only fields entering the momentum and turbulence transport equations carry information forward.

## 7.3 The wake is worth +0.5% here

The largest acceleration in this literature initialises the far wake (arXiv:2501.14699; 26.3x iterations). Every seed here does the opposite. The obvious next move is to compose the two — so we bounded the payoff before building anything.

`scripts/wake_probe.py` seeds the **exact converged field** across the whole downstream region (37.5% of cells, 21.6% of them fully), which bounds what any wake model could ever buy on these cases. Five cases,  $Re = 3e6$ :

| metric         | oracle wake seed | 95% CI       | per case                  |
|----------------|------------------|--------------|---------------------------|
| <b>Cd_v@1%</b> | <b>+0.5%</b>     | [+0.4, +0.7] | +0, +0, +1, +1, +1        |
| Cd_v@0.5%      | +1.3%            | [+1.0, +1.5] | +1, +1, +1, +1, +2        |
| Cd@1%          | -242.1%          | [-676, -0.3] | -1098, -103, -19, -4, +13 |
| Cl@1%          | -22.0%           | [-68, +1.6]  | -92, +0, +2, +2           |

**The perfect wake seed is worth half a percent.** On a 2-D attached-flow airfoil at  $Re = 3e6$  on a 20-chord C-grid, the solver is not spending its time developing the wake; it is spending it on the near-wall state, exactly where §7.2 located it. The 26.3x figure is therefore a fact about that configuration, geometry and cold baseline — not a better method — and our restriction to the boundary layer is a finding rather than a compromise.

Note also that the oracle wake seed is *harmful* on total drag. Handing over a downstream field while leaving the boundary layer cold is another inconsistent pair, which is §5.4’s lesson at a length scale two orders of magnitude larger.

## 8. An acceptance test that bounds the worst case

Warm starting is only adoptable if a bad seed cannot cost more than not seeding. Ours can: ungated across 5 cases x 15 strategies (70 seeds, every arm in the tree, not a favourable subset), the mean is -163.6% on Cd@1% and the worst single seed is -1169.6%. Only 24 of the 70 seeds help at all.

**The rule.** Run K probe iterations from the seed. Read two scalars from the residual history — the level  $\log_{10} r_K$  and the drop  $\log_{10} r_K - \log_{10} r_0$ . Either continue the solve, or discard the seed and start cold. Accepting costs nothing extra, because the probe iterations *are* the first K iterations of the warm solve; rejecting costs K + cold. The worst case is therefore  $(1 + K/N_{\text{cold}}) \times \text{cold}$  **by construction**, whatever the seed does.

**The rule never observes a cold run.** In production there isn’t one — that is the entire point of warm starting. The threshold is calibrated leave-one-case-out and applied to the held-out case.

$K = 25$  (~3% of a cold solve), threshold on the residual level, scored over every arm in the `repr3` tree:

| metric        | seeds | ungated mean | ungated worst | <b>gated mean</b> | <b>gated worst</b> | harmful admitted |
|---------------|-------|--------------|---------------|-------------------|--------------------|------------------|
| Cd@1%         | 70    | -163.6%      | -1169.6%      | <b>+1.5%</b>      | <b>-5.8%</b>       | <b>0 / 46</b>    |
| residual 5e-6 | 70    | -161.9%      | -1449.3%      | -0.5%             | <b>-7.6%</b>       | <b>0 / 53</b>    |
| Cd_v@1%       | 70    | +20.1%       | -8.6%         | +20.1%            | -8.6%              | 14 / 15          |
| Cl@1%         | 56    | +1.1%        | -672.6%       | <b>-8.2%</b>      | <b>-672.6%</b>     | 5 / 18           |

**Read the two columns that matter as a pair.** The gated mean is small — the gate is insurance, not a profit centre, and selling it as a mean saving would be selling the wrong product. What it does is convert a -1169.6% tail into a -5.8% one on drag, and a -1449.3% tail into -7.6% on the residual, while admitting **none** of the harmful seeds in either case. On viscous drag, where 55 of 70 seeds already help, it is a near no-op at 96% capture.

**Where the gate fails, stated rather than buried.** On lift it admits 5 of 18 harmful seeds, and its gated worst case is -672.6% — the *same* as the ungated worst, meaning it admits the single worst lift seed in the study. Lift converges by a different route — it is pressure-dominated, and §5.2 showed that resampling *helps* lift while destroying drag — so a probe reading the momentum residual is weakly informative about it. The gate should be applied per quantity, and

on the quantity a user cares about; we do not claim it is a universal filter. Its worst-case bound of  $(1 + K/N) \times \text{cold}$  survives regardless, because that bound is arithmetic, not statistical: it holds for any seed, any metric, and any threshold, including a threshold that admits every seed.

The gate is not what makes warm starting fast; it is what makes it deployable. It is conservative by construction, capturing only 12% of what a gatekeeper with foreknowledge would achieve on total drag — the metric where two thirds of the seeds are harmful. Longer probes are monotonically worse, and not marginally: at  $K = 400$  on drag the rule admits 13 harmful seeds it rejected at  $K = 25$  and returns  $-43.9\%$ , because the probe cost alone (bound  $-49.8\%$ ) exceeds anything the decision can recover. **A short probe is not a compromise forced by cost; it is the better rule.**

## 9. Wall-clock

Iterations are the honest unit for a mechanism, but the claim an engineer cares about is seconds — and a mesh-native seed costs seconds a projected one does not: wall-distance computation, surrogate inference, masking, and writing the case.

`scripts/wallclock_control.py` charges each preparation stage to the arms that need it and reports **end-to-end** seconds. It runs serially and refuses to start while any other solver is up; that refusal is part of the measurement. Five cases, all arms, **exclusive: true**.

| arm                   | iterations    | end-to-end seconds |
|-----------------------|---------------|--------------------|
| oracle_mesh (control) | +92.0%        | <b>+93.1%</b>      |
| nf_bl                 | <b>+34.0%</b> | <b>+28.8%</b>      |
| fitted_bl             | -190.7%       | -138.0%            |
| cartesian_128         | -513.5%       | -308.4%            |

**The iteration saving survives translation into seconds, and the cost of the translation is about five percentage points.** That is the transferable finding here, and it holds case by case rather than only on the mean:

| case       | iterations | seconds | gap |
|------------|------------|---------|-----|
| naca001204 | -3.3%      | -3.4%   | 0.1 |
| naca241202 | +40.9%     | +34.7%  | 6.2 |
| naca001506 | +27.6%     | +21.5%  | 6.1 |
| naca001200 | +40.1%     | +34.5%  | 5.6 |
| naca241505 | +64.9%     | +56.9%  | 8.0 |

Seed construction is charged in full and is small against the solve: backbone inference at 31,700 points **10.4–10.6 s**, wall\_distance 0.4 s, masking 0.4 s — **~11 s** against cold solves of 88–239 s. The rest of the five-point gap is the per-iteration penalty of a seeded solve, which is modest.

Two details worth stating because they cut *for* us and could look like errors: the **oracle does better in seconds than in iterations** (+93.1% vs +92.0%), and the Cartesian arm is **less bad** in seconds than in iterations ( $-308\%$  vs  $-514\%$ ). Both have the same cause — a solve started near the answer has cheaper inner linear solves — and both mean iteration counts are the conservative unit.

**Readability, stated rather than assumed.** This tree carries five arms, not the fifteen of the mechanism study, so its reference is a median over fewer settled arms and its spreads are wider:

two of the five cases (`naca2412@2` at 0.751%, `naca0012@0` at 0.745%) exceed the 0.5% limit for a 1% band. On the three readable cases the result is **+29.7% iterations**  $\rightarrow$  **+25.0% seconds**, a 4.8-point translation cost — the same conclusion at a smaller  $n$ . The per-case gap column above is what we actually lean on, and it is independent of the readability verdict entirely.

With this, the last clause of our pre-registered bar — *confirmed in wall-clock seconds, inference and seed construction included* — is **met**.

## 10. Limitations

- **2-D, incompressible, steady, one turbulence model.** Spalart–Allmaras only. Whether the three conditions survive a two-equation model is untested [[B2]].
- **One solver and one mesh family.** OpenFOAM SIMPLEC on a C-grid we generate. Nothing here has been tried on an unstructured or commercial solver.
- **One Reynolds number for the headline.**  $Re = 3e6$ ; the moderate- $Re$  result is  $n = 1$  per Reynolds number and one low- $Re$  claim is withdrawn pending re-measurement.
- **Bands below 1% need a longer budget.** At  $b = 0.5\%$  and  $0.2\%$  the total-drag rows are unreadable at 6000 iterations on the corpus, and on the mechanism study the  $0.5\%$  margin is  $0.232\%$  against a  $0.25\%$  limit [[B4]].
- **nuTilda is floored at freestream** on write, a common-mode limitation quantified in §5.5.
- **Three cases have no unique steady drag at this budget**, and they share a shape: `naca4412@3°`, `naca4415@2°` and `naca4415@4°` — thick cambered sections at low incidence. Their arms settle on drag values 1.0–7% apart while every other case in either study agrees to  $\leq 0.113\%$ , and they carry the corpus’s worst residual floors ( $8.75e-6$ ,  $1.6e-5$ ,  $1.22e-6$ , against  $\sim 1e-7$  elsewhere). `4412@3` is excluded with cause from the mechanism study; the two `4415` cases are **admitted and reported** in the corpus, where they cost us every total-drag row (§5.1). We do not know whether this is genuine non-uniqueness or a budget we did not pay, and we do not claim to.
- **The headline misses our own pre-registered threshold:**  $+18.4\%$  against  $+30\%$  (§5.1). Stated rather than restated.
- **The residual metric is negative for the recommended arm** (§5.6).
- **Wall-clock is  $n = 5$ , and two of those five exceed the readability limit on that tree** (§9). The per-case iteration-to-seconds gap, which is what we lean on, is consistent across all five and independent of that verdict.

## 11. Conclusion

A neural surrogate can accelerate a production RANS solver — **+18.4% on viscous drag, on every one of thirteen cases from attached flow into incipient separation,  $p = 0.0002$**  — and the conditions under which it does are narrow, checkable, and explicable by a single measured quantity. It must be evaluated at the solver’s own cell centres, because the first-cell wall gradient — which no practical grid representation can carry — is what viscous drag integrates. It must hand over the boundary layer and nothing else, because the outer field is where the model extrapolates. And it must hand over velocity and eddy viscosity together, because a turbulence field inconsistent with its own strain is worse than no turbulence field at all.

Read as advice rather than as a result, the paper is short: **query your surrogate where the solver lives, give it only the region your surrogate was trained on, give it whole physics rather than single channels, and spend 3% of a solve checking before you commit.**

The size of the effect is the part we would revise downward if we ran this again. We pre-registered +30% and measured +18.4%; the +33.9% on total drag that the five-case study supports does not survive to thirteen cases as a readable number. What did survive is the shape of the result — every case, both controls behaving, monotone across bands — and the mechanism that explains it, which is stated in units a reader can check on their own mesh before running anything: **how much of the first-cell wall gradient does your representation keep?** Ours keeps 46% of it and buys 18%. A 16,384-value grid keeps none of it and costs you the solve.

## Appendix A — reproduction

Every table above is regenerated by one command from a checkpointed tree; see `docs/PLANS.md` §3 for the mapping from table to script and `results/*.json` file.

## Appendix B — the scoring rules as code

`solver/scoring.py`: `has_settled`, `settled_reference`, `bounded_saving`, `shared_reference`, `reference_spread`, `readable_depth`, `bootstrap_ci`, `sign_test`, with `MIN_DEPTH_OVER_FLOOR` = 5.0 and `MAX_SPREAD_FRACTION` = 0.5. Each has a test named for the mistake it prevents.